

JAVA DEVELOPER INTERNSHIP

TASK 2

STUDENT MANAGEMENT SYSTEM

OBJECTIVE:

The objective of this task is to develop a console-based Student Management System using Java. The project demonstrates the use of classes, objects, ArrayList, Scanner, loops, and conditional statements to perform operations such as adding, viewing, searching, and deleting student records.

TOPICS COVERED:

- Classes and Objects
- Constructors
- ArrayList
- Scanner Class
- User Input Handling
- Loops
- Conditional Statements
- Switch Case
- Search Operation
- Delete Operation
- Menu-Driven Programming
- Object-Oriented Programming Concepts

FEATURES IMPLEMENTED:

1. Add Student

Allows the user to enter student details such as ID, name, and course and store them in the system.

2. View Students

Displays all the student records available in the system.

3. Search Student

Allows the user to search for a student using the student ID.

4. Delete Student

Enables the user to remove a student record from the system.

5. Exit Program

Terminates the application safely.

The Student Management System provides a menu-driven interface that allows users to perform various operations on student records. It demonstrates the use of Java fundamentals and Object-Oriented Programming concepts to manage data efficiently.

SOURCE CODE:

StudentManagementSystem.java

```
import java.util.ArrayList;

import java.util.Scanner;

class Student

{

    int id;

    String name;

    String course;

    Student(int id, String name, String course)

{

    this.id = id;

    this.name = name;

    this.course = course;

}

    void display()

{

    System.out.println("ID: " + id);

    System.out.println("Name: " + name);

    System.out.println("Course: " + course);

    System.out.println("-----");

}

}
```

```
public class StudentManagementSystem

{

    public static void main(String[] args)

    {

        ArrayList<Student> students = new ArrayList<>();

        Scanner sc = new Scanner(System.in);

        int choice;

        do

        {

            System.out.println("\n===== STUDENT MANAGEMENT SYSTEM =====");

            System.out.println("1. Add Student");

            System.out.println("2. View Students");

            System.out.println("3. Search Student");

            System.out.println("4. Delete Student");

            System.out.println("5. Exit");

            System.out.print("Enter your choice: ");

            choice = sc.nextInt();

            switch (choice)

            {

                case 1:

                    System.out.print("Enter Student ID: ");

                    int id = sc.nextInt();

                    sc.nextLine();
```

```
System.out.print("Enter Student Name: ");

String name = sc.nextLine();

System.out.print("Enter Course: ");

String course = sc.nextLine();

students.add(new Student(id, name, course));

System.out.println("Student Added Successfully!");

break;
```

case 2:

```
if (students.isEmpty())
```

```
{
```

```
    System.out.println("No students found.");
```

```
}
```

else

```
{
```

```
    for (Student s : students)
```

```
{
```

```
        s.display();
```

```
    }
```

```
}
```

```
break;
```

case 3:

```
System.out.print("Enter Student ID to Search: ");
```

```
int searchId = sc.nextInt();
```

```
boolean found = false;

for (Student s : students)

{

    if (s.id == searchId)

    {

        s.display();

        found = true;

    }

}

if (!found)

{

    System.out.println("Student not found.");

}

break;

case 4:

    System.out.print("Enter Student ID to Delete: ");

    int deleteId = sc.nextInt();

    boolean removed = students.removeIf(s -> s.id == deleteId);

    if (removed)

    {

        System.out.println("Student Deleted Successfully!");

    }

}
```

```
else
{
    System.out.println("Student not found.");
}

break;

case 5:

    System.out.println("Exiting Program...");

    break;

default:

    System.out.println("Invalid Choice!");

}

}

while (choice != 5);

    sc.close();

}

}
```

Code Explanation:

The Student Management System is a menu-driven application developed using Java. It uses classes and objects to represent student records and ArrayList to store data dynamically. The Scanner class is used to accept user input, while loops and switch-case statements are used to perform operations such as adding, viewing, searching, and deleting student records.

OUTPUT:

```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-2-Student-Management-System>javac StudentManagementSystem.java

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-2-Student-Management-System>java StudentManagementSystem

===== STUDENT MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice:
```

```
C:\Windows\System32\cmd.e X + v
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-2-Student-Management-System>javac StudentManagementSystem.java

C:\Users\DEEKSHITH\OneDrive\Pictures\Documents\Desktop\srihani\Task-2-Student-Management-System>java StudentManagementSystem

===== STUDENT MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: 1
Enter Student ID: 701
Enter Student Name: Honey
Enter Course: CSE
Student Added Successfully!
```

```
===== STUDENT MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: 2
ID: 701
Name: Honey
Course: CSE
-----
```

```
===== STUDENT MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: 3
Enter Student ID to Search: 701
ID: 701
Name: Honey
Course: CSE
-----
```

```
===== STUDENT MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: 4
Enter Student ID to Delete: 701
Student Deleted Successfully!

===== STUDENT MANAGEMENT SYSTEM =====
1. Add Student
2. View Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: 5
Exiting Program...
```

The Student Management System was executed successfully. The program allows users to add, view, search, and delete student records through a menu-driven interface. All operations were performed correctly, demonstrating the proper implementation of Java programming concepts.

CONCEPTS USED:

Classes and Objects

Classes are blueprints used to create objects. Objects are instances of classes that represent real-world entities and allow access to their properties and methods.

ArrayList

ArrayList is a dynamic collection framework class used to store and manage student records efficiently. It allows elements to be added and removed dynamically.

Scanner Class

The Scanner class is used to accept input from the user through the keyboard. It helps in reading different types of data during program execution.

Loops

Loops are used to repeatedly display the menu and execute operations until the user chooses to exit the program. They reduce code repetition and improve efficiency.

Conditional Statements

Conditional statements and switch-case are used to perform different operations based on the user's choice. They help control the flow of execution.

Menu-Driven Programming

Menu-driven programming provides an interactive interface where users can select different options to perform various operations. It improves usability and makes the program user-friendly.

Object-Oriented Programming

Object-Oriented Programming helps in organizing code using classes and objects. It promotes code reusability, modularity, and maintainability.

CONCLUSION:

The Student Management System project provided practical experience in developing a menu-driven application using Java. Through the implementation of various operations such as adding, viewing, searching, and deleting student records, important programming concepts including classes, objects, ArrayList, Scanner, loops, and conditional statements were applied successfully. This task enhanced problem-solving abilities and strengthened the understanding of Object-Oriented Programming concepts and Java application development.